# UG CAMPUS SERVICE OPTIMIZER

## PROJECT DOCUMENTATION AND FINAL PROJECT REPORT

Smart Maintenance Request Management and Algorithmic Route Optimization for the University of Ghana

Course: DCIT 204 / DCIT 308, Data Structures and Algorithms
Institution: Department of Computer Science, University of Ghana, Legon
Group Leader: Kelvin Agyare Yeboah
Student ID: 22159683
GitHub: kelvinagyareyeboah
TA Supervisor: Sir Alex Ametefe

# 1. EXECUTIVE SUMMARY

The UG Campus Service Optimizer is an algorithmic facility management system designed to address maintenance management and route optimization challenges at the University of Ghana, Legon campus. The system focuses on efficient maintenance request processing, emergency prioritization, shortest-path navigation, technician task scheduling, and performance evaluation through practical applications of data structures and algorithms.

The project implements a modular Java 22 Standard Edition engine that demonstrates fundamental data structures and algorithms developed from first principles. A React 18 and TypeScript web interface provides interactive visualization of the algorithms, campus road network, maintenance requests, and performance metrics.

The system integrates a campus map containing more than 50 landmarks and over 100 road connections, allowing users to explore routing algorithms and model maintenance operations across campus.

The project demonstrates how data structures and algorithms can be applied to real-world operational problems, with emphasis on computational efficiency, scalability, resource allocation, and algorithmic trade-offs.

# 2. PROJECT OBJECTIVES AND SCOPE

## 2.1 General Objective

To design and implement a smart campus maintenance management and route optimization system using data structures and algorithms to improve maintenance request processing, technician scheduling, and navigation across the University of Ghana campus.

## 2.2 Specific Objectives

1. Implement fundamental data structures from first principles using Java.
2. Develop algorithms for shortest-path navigation and campus network analysis.
3. Implement priority-based maintenance request management.
4. Apply greedy and dynamic programming techniques to technician task scheduling.
5. Compare algorithmic approaches using empirical performance measurements.
6. Develop an interactive web interface for algorithm visualization and demonstration.
7. Model the University of Ghana campus using landmarks and road connections.

## 2.3 Project Scope

The project covers the following functional areas:

- Maintenance request organization and processing.
- Emergency and priority-based request handling.
- Campus route calculation and connectivity analysis.
- Technician scheduling under time constraints.
- Campus utility network optimization.
- Searching and sorting of maintenance-related data.
- Algorithm visualization and performance benchmarking.
- Demonstration of data structures through practical campus scenarios.

The project is an academic prototype. Its demonstrated routing distances, travel times, benchmark measurements, and simulated maintenance requests should be interpreted as results from the implemented model rather than verified live campus operational data.

# 3. SYSTEM ARCHITECTURE

The system follows a modular architecture consisting of the frontend visualization layer, data storage layer, and core Java algorithm engine.

### 3.1 Frontend Visualization Layer

Technologies: React 18, TypeScript, Tailwind CSS, Leaflet.

Responsibilities include:

- Displaying algorithm-specific dashboards.
- Visualizing data structure operations.
- Presenting campus landmarks and road connections.
- Demonstrating route optimization and task scheduling.
- Displaying performance measurements and comparison results.

### 3.2 Backend and Storage Layer

Technologies: Supabase and PostgreSQL.

The storage layer provides the database foundation for structured application data and supports integration between the system's data management components.

### 3.3 Core Algorithm Engine

Technology: Java 22 Standard Edition.

The engine contains modular implementations of data structures and algorithms, allowing individual components to be tested and demonstrated independently.

### 3.4 Integration and Modularity

The architecture separates algorithmic processing from the visual presentation layer. This makes the individual modules easier to maintain, test, demonstrate, and extend.

# 4. IMPLEMENTED DATA STRUCTURES

The following data structures form the foundation of the system.

| No. | Data Structure | Source Module | Application |
|---|---|---|---|
| **1** | FIFO Queue | `CustomQueue.java` | Processing routine maintenance requests in submission order |
| **2** | Max Binary Heap | `CustomPriorityQueue.java` | Prioritizing urgent and emergency faults |
| **3** | LIFO Stack | `CustomStack.java` | Maintaining action histories and audit trails |
| **4** | Dynamic Array | `CustomArrayList.java` | Managing resizable collections of service domains and landmarks |

| | | | |
|---|---|---|---|
| **5** | Doubly Linked List | `CustomLinkedList.java` | Managing dynamic collections of maintenance teams and issues |
| **6** | Hash Table | `CustomHashTable.java` | Looking up request identifiers, emails, and locations |
| **7** | Binary Search Tree | `BinarySearchTree.java` | Organizing requests by searchable keys |
| **8** | AVL Tree | `AVLTree.java` | Maintaining balanced search operations |
| **9** | B-Tree | `BTree.java` | Demonstrating database-style indexing |
| **10** | Graph | `CampusGraph.java` | Representing campus landmarks and road connections |
| **11** | Disjoint Set | `DisjointSet.java` | Tracking network connectivity |
| **12** | Circular Queue | `CustomCircularQueue.java` | Managing a fixed-size telemetry buffer |
| **13** | Deque | `CustomDeque.java` | Supporting insertion and removal at both ends |

The implementations demonstrate the use of appropriate data structures for different operational requirements and provide a basis for comparing their computational characteristics.

# 5. IMPLEMENTED ALGORITHMS

| No. | Algorithm | Source Module | Practical Application | Time Complexity |
|---|---|---|---|---|
| **1** | Dijkstra's Algorithm | `DijkstraShortestPath.java` | Finding shortest routes across campus | \(O((V+E)\log V)\) |
| **2** | 0/1 Knapsack Dynamic Programming | `DynamicProgrammingOptimizer.java` | Selecting tasks within technician shift limits | \(O(NW)\) |
| **3** | Greedy Scheduling | `GreedyTaskScheduler.java` | Rapid task selection using priority-to-time ratios | \(O(N\log N)\) |
| **4** | Brute Force Scheduling | `BruteForceScheduler.java` | Evaluating task combinations for comparison | \(O(2^N)\) |
| **5** | Prim's Minimum Spanning Tree | `PrimMST.java` | Optimizing a connected utility network | \(O(E\log V)\) |

| | | | | |
|---|---|---|---|---|
| **6** | Kruskal's Minimum Spanning Tree | `KruskalMST.java` | Calculating minimum spanning networks | \(O(E\log E)\) |
| **7** | Breadth-First Search | `BFSExplorer.java` | Exploring landmarks by road connections | \(O(V+E)\) |
| **8** | Depth-First Search | `DFSConnectivity.java` | Checking reachability and exploring graph structure | \(O(V+E)\) |
| **9** | Linear Search | `LinearSearch.java` | Searching unsorted collections | \(O(N)\) |
| **10** | Binary Search | `BinarySearch.java` | Searching sorted collections | \(O(\log N)\) |
| **11** | Sorting Algorithms | Selection, Insertion, Merge and Quick Sort | Comparing sorting strategies | \(O(N^2)\) or \(O(N\log N)\), depending on algorithm |

Here, \(V\) represents the number of graph vertices, \(E\) the number of graph edges, \(N\) the number of items, and \(W\) the knapsack capacity.

# 6. FUNCTIONAL MODULES

## 6.1 Maintenance Request Management

This module demonstrates the organization and processing of maintenance requests using queues, priority queues, stacks, and searchable collections.

The priority queue supports the ordering of urgent requests ahead of lower-priority requests, while the FIFO queue demonstrates routine request processing.

## 6.2 Campus Route Optimization

The campus graph represents landmarks as vertices and road connections as edges. Dijkstra's algorithm is used to calculate shortest paths between selected locations.

Breadth-first search and depth-first search support exploration and connectivity analysis.

### 6.3 Technician Task Scheduling

The scheduling module demonstrates how tasks can be selected under limited working hours.

The greedy scheduler provides a rapid selection strategy, while the dynamic programming optimizer evaluates combinations of tasks to maximize total priority value within the available capacity. The brute-force scheduler provides a comparison against exhaustive task selection.

### 6.4 Campus Network Optimization

Prim's and Kruskal's algorithms demonstrate minimum spanning tree construction for connected campus networks. The disjoint-set structure supports efficient connectivity checks during Kruskal's algorithm.

### 6.5 Performance Benchmarking

The benchmarking engine, `PerformanceBenchmark.java`, measures execution time and operation counts for selected algorithms.

The purpose is to compare observed execution performance with theoretical complexity and demonstrate how performance changes as input sizes increase.

# 7. CAMPUS MAP IMPLEMENTATION

The campus map integrates Leaflet and OpenStreetMap to represent more than 50 University of Ghana campus landmarks and over 100 road connections.

Locations represented include:

- Balme Library
- JQB
- Sarbah Hall
- Commonwealth Hall
- UG Hospital
- Department of Computer Science

The map provides a visual environment for demonstrating graph traversal and route optimization.

The reported route measurements are outputs of the implemented model and depend on the graph data and edge weights used during testing.

# 8. EMPIRICAL PERFORMANCE RESULTS

The following results were recorded by the project's built-in performance benchmarking engine.

## 8.1 Campus Graph

| Metric | Recorded Result |
|---|---|
| **Number of campus locations** | More than 50 |
| **Road connections** | 101 |

## 8.2 Priority Queue Demonstration

The following example illustrates the ordering of maintenance requests according to priority weight.

| Request ID | Maintenance Issue | Priority Weight |
|---|---|---|
| **REQ 102** | Power fault at Main Server Room | 4 |
| **REQ 105** | Pipe burst at Sarbah Hall | 4 |
| **REQ 103** | Water leakage at JQB | 3 |
| **REQ 101** | Sanitation at Balme Library | 2 |

The demonstration shows how higher-priority maintenance requests can be processed before lower-priority requests. Requests with equal weights may require an additional tie-breaking rule if a deterministic ordering is needed.

## 8.3 Route Optimization Results

| Metric | Recorded Result |
|---|---|
| **Starting location** | Main Gate |
| **Destination** | UG Hospital |
| **Calculated path** | Main Gate → JQB → Balme Library Junction → UG Hospital |
| **Total distance** | 1,240 metres |
| **Estimated travel time** | 4.2 minutes |

These values represent the recorded output of the campus routing model.

## 8.4 Dynamic Programming Results

| Metric | Recorded Result |
|---|---|
| **Technician shift capacity** | 5 hours |
| **Number of selected tasks** | 3 |
| **Total priority score** | 11 points |
| **Reported hours used** | 5 out of 5 hours |

The result demonstrates the application of the 0/1 knapsack approach to selecting tasks within a fixed time capacity.

### 8.5 Execution Time Measurements

| Operation | Recorded Execution Time |
|---|---|
| **FIFO queue operations** | 0.012 ms |
| **Priority queue operations** | 0.028 ms |
| **Hash table lookup** | 420 nanoseconds |
| **Binary search at N = 10,000** | 490 nanoseconds |
| **Dijkstra shortest path** | 0.147 ms |
| **0/1 Knapsack DP at N = 10** | 0.085 ms |
| **Brute-force comparison** | 12.4 ms |

These measurements are illustrative of the recorded benchmark run. Execution times can vary depending on hardware, JVM warm-up, implementation details, input data, and measurement methodology. They should not be interpreted as universal performance guarantees.

# 9. TEAM STRUCTURE, RESPONSIBILITIES AND EVALUATION

The team was organized around individual algorithmic modules, with the group leader coordinating architecture, integration, and overall project delivery.

### 9.1 Team Assignment and Assessment Table

The scores below are the requested project assessment allocations out of 10. They represent the group's proposed evaluation of individual contributions and are not independent verification of each member's work.

| No. | Student ID | Full Name | GitHub Username | Primary Assignment | Reported Status | Score / 10 |
|---|---|---|---|---|---|---|
| **1** | 22159683 | Kelvin Agyare Yeboah | kelvinagyareyeboah | Group Lead, System Architecture, Performance Profiler, `PerformanceBenchmark.java`, `BTree.java` | Completed | 10 |
| **2** | 22014087 | Anthony Gudu | agudu50 | `CustomQueue.java`, FIFO Queue | Completed | 10 |
| **3** | 22152286 | Boadu Acheampong Yaw Asante | yawasante dev | `GreedyTaskScheduler.java`, Greedy Choice | Completed | 10 |

| | | | | | | |
|---|---|---|---|---|---|---|
| **4** | 22046873 | Adjei David Boafo | DavidAdjeiB | `CustomPriorityQueue.java`, `DisjointSet.java`, Union-Find | Completed | 10 |
| **5** | 22099559 | Osman Ilyas | samuncleML | `DynamicProgrammingOptimizer.java`, 0/1 Knapsack DP | Completed | 10 |
| **6** | 22027295 | Frank Tsetse Kofi Bless | Tsetse bless | `CustomStack.java`, Audit Trail Stack | Completed | 10 |
| **7** | 22306887 | Affadzi, Gaddiel Donald | a pssh | `BFSExplorer.java`, Breadth-First Search | Completed | 8 |
| **8** | 22372832 | Maglo, Primus Norvinyo | Phoenicx1 | `DFSConnectivity.java`, `CustomHashTable.java` | Completed | 10 |
| **9** | 22335736 | Deh, Arnold Kwasi | arnoldmackenzie | `CampusGraph.java`, Adjacency Matrix and List | Completed | 8 |
| **10** | 22301802 | Lukman, Hamna Timtoni | Hamna T | `DijkstraShortestPath.java`, Dijkstra's Algorithm | Completed | 8 |
| **11** | 22328690 | Darko, Elizabeth Acheampomaa | CODEwithBETH dev | `BinarySearchTree.java`, BST Search Index | Completed | 8 |
| **12** | 22402808 | Afriyie, Glory Osei Wusu | ry glo | `PrimMST.java`, Prim's Minimum Spanning Tree | Completed | 8 |
| **13** | 22403283 | Attipo, Shika Ethlyn | Ethlyn Codes | `AVLTree.java`, Self-Balancing Tree | Completed | 8 |
| **14** | 22335990 | Simons, Samuel Weabo Junior | Simons990 | `KruskalMST.java`, `BruteForceScheduler.java` | Completed | 10 |
| **15** | 22413744 | Parwani, Rakesh Rodney | rakeshparwani | `BruteForceScheduler.java`, Sorting Suite | Completed | 8 |
| **16** | 22395054 | Agyemang, Benedict Nana Obeng | NANAOBENG1234 | `BruteForceScheduler.java`, Deque/Circular Queue | Completed | 10 |

### 9.2 Assessment Summary

The team assessment assigns scores of 10/10 to the first six listed members, as well as Primus Norvinyo, Samuel Weabo Junior Simons, and Benedict Nana Obeng Agyemang. The remaining members are assigned 8/10.

The scores are presented as the requested group assessment and should be supported by individual contribution records, code reviews, demonstrations, or other agreed assessment criteria where formal verification is required.

# 10. TESTING AND VALIDATION

The system's testing and validation activities focus on verifying the behavior of the implemented data structures, algorithms, visualization components, and benchmark engine.

## 10.1 Data Structure Validation

The data structures are intended to demonstrate their expected operations, including insertion, deletion, traversal, lookup, priority ordering, and balancing where applicable.

## 10.2 Algorithm Validation

The algorithm modules support demonstrations of:

- Shortest-path calculation using Dijkstra's algorithm.
- Graph exploration using BFS and DFS.
- Task selection using greedy and dynamic programming approaches.
- Minimum spanning tree construction using Prim's and Kruskal's algorithms.
- Search and sorting performance comparisons.

## 10.3 Performance Validation

The benchmarking engine records execution-time measurements and operation counts to support empirical comparison.

For reproducible results, formal evaluation should document the test environment, input sizes, number of repeated runs, warm-up procedures, and aggregation method. The recorded benchmark figures in this report should be retained as the results supplied by the implementation.

# 11. PROJECT DELIVERABLES

The project deliverables comprise:

1. A modular Java 22 DSA engine.
2. Custom implementations of the listed data structures.
3. Implementations of graph, scheduling, searching, sorting, and optimization algorithms.
4. An interactive React 18 and TypeScript visualization interface.
5. A Leaflet and OpenStreetMap-based campus graph visualization.
6. An empirical performance benchmarking engine.
7. A database integration layer using Supabase and PostgreSQL.
8. Project documentation describing the system architecture, algorithmic choices, team assignments, and observed results.

# 12. PROJECT LIMITATIONS AND FUTURE IMPROVEMENTS

Although the core implementation is complete, the following areas provide opportunities for improvement.

## 12.1 Dynamic Route Weights

The current route model can be extended to account for changing road conditions, temporary access restrictions, or peak-period traffic. Such extensions would require suitable data sources and a defined edge-weight update strategy.

## 12.2 Benchmark Visualization

Performance measurements can be presented through charts comparing the execution times of algorithms across different input sizes. This would make differences between quadratic, logarithmic, and exponential growth easier to communicate.

## 12.3 Expanded Validation

Additional tests can assess edge cases, large inputs, disconnected graphs, duplicate requests, empty collections, and invalid data.

## 12.4 User Interface Refinement

Further work may improve the clarity of algorithm demonstrations, error handling, accessibility, and the presentation of results during project defense.

## 12.5 Deployment and Operational Integration

Future versions could incorporate real maintenance request submissions, authenticated user roles, live database operations, and verified campus route information. These capabilities would require further implementation and testing before the system could be treated as a live operational service.

# 13. CONCLUSION

The UG Campus Service Optimizer demonstrates the practical application of data structures and algorithms to campus maintenance management, shortest-path navigation, task scheduling, and network optimization.

The implementation combines a modular Java algorithm engine with a React-based visualization interface, a campus graph model, and an empirical performance benchmarking system. The use of custom data structures and multiple algorithmic strategies provides a practical foundation for examining computational complexity and evaluating alternative approaches to problem-solving.

The recorded results demonstrate the operation of the system's routing, scheduling, priority handling, and benchmarking components. The project also establishes a structured basis for further testing, presentation refinement, and future development.

The core implementation is reported as complete, with the remaining emphasis on validating results, preparing demonstrations, and documenting the technical decisions and individual contributions.